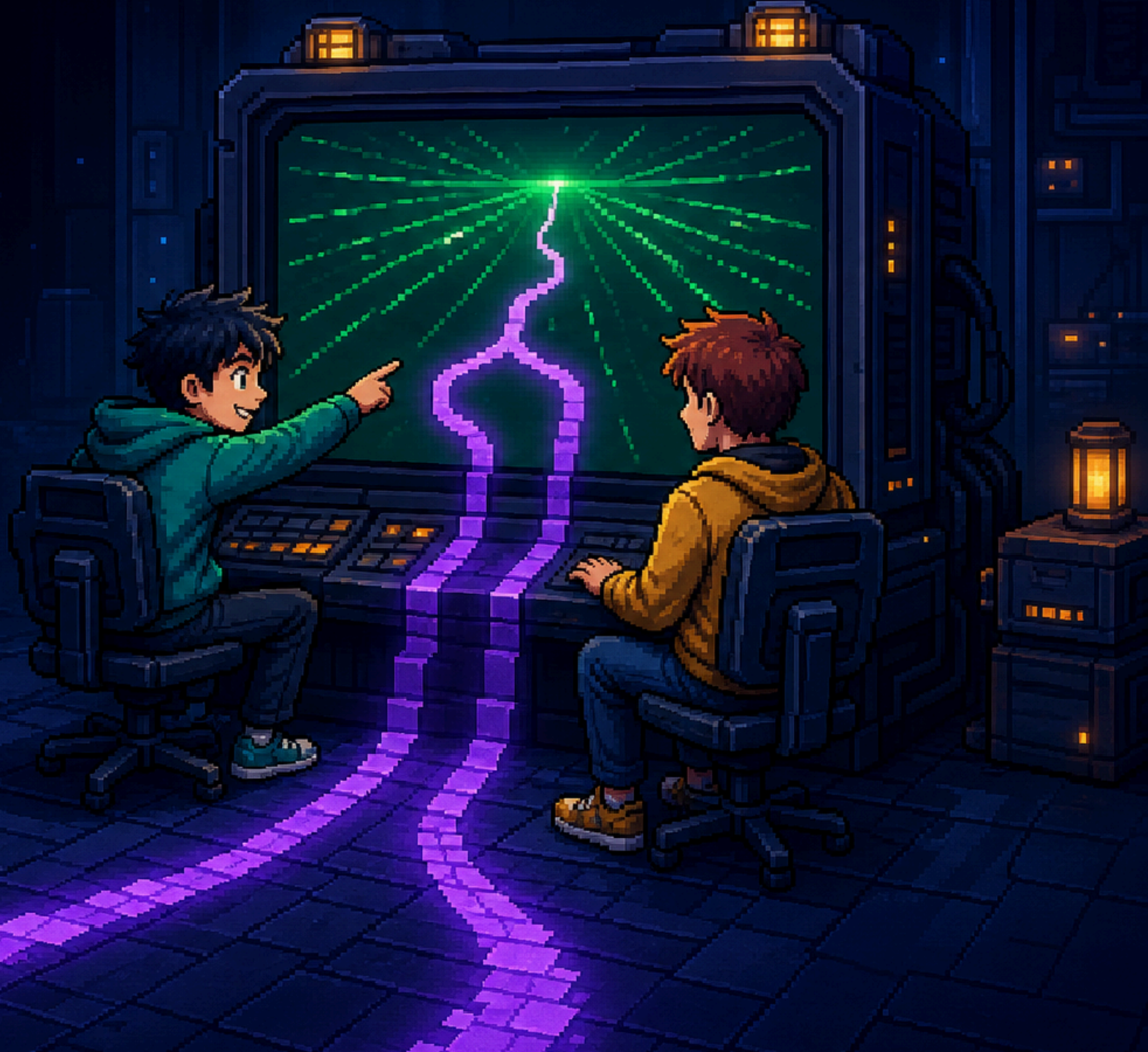
Git 协作四关

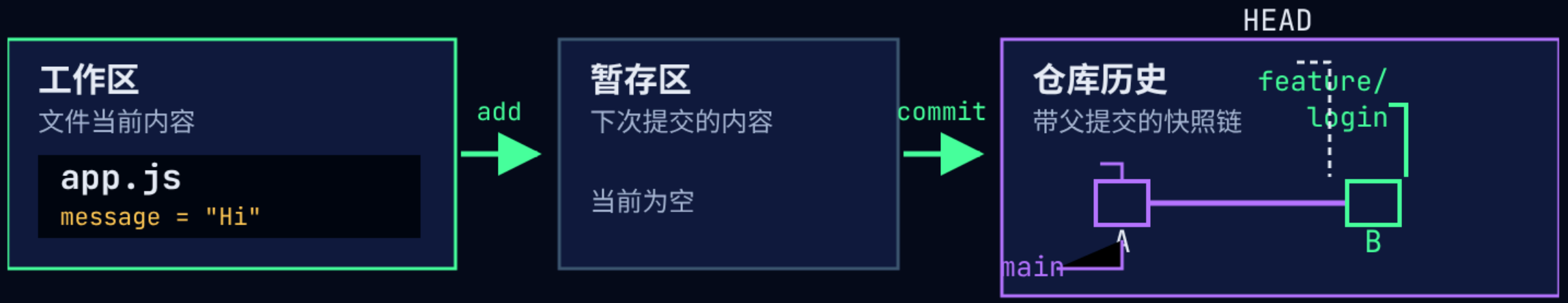
冲突 · 变基 · 暂存 · 游离 HEAD

看见文件、分支与历史如何变化



先看清工作区、分支与历史

教学设定：仓库 shop-app，当前在 feature/login，app.js 有未暂存修改。



A：初始化应用

B：加入登录按钮（父提交：A）

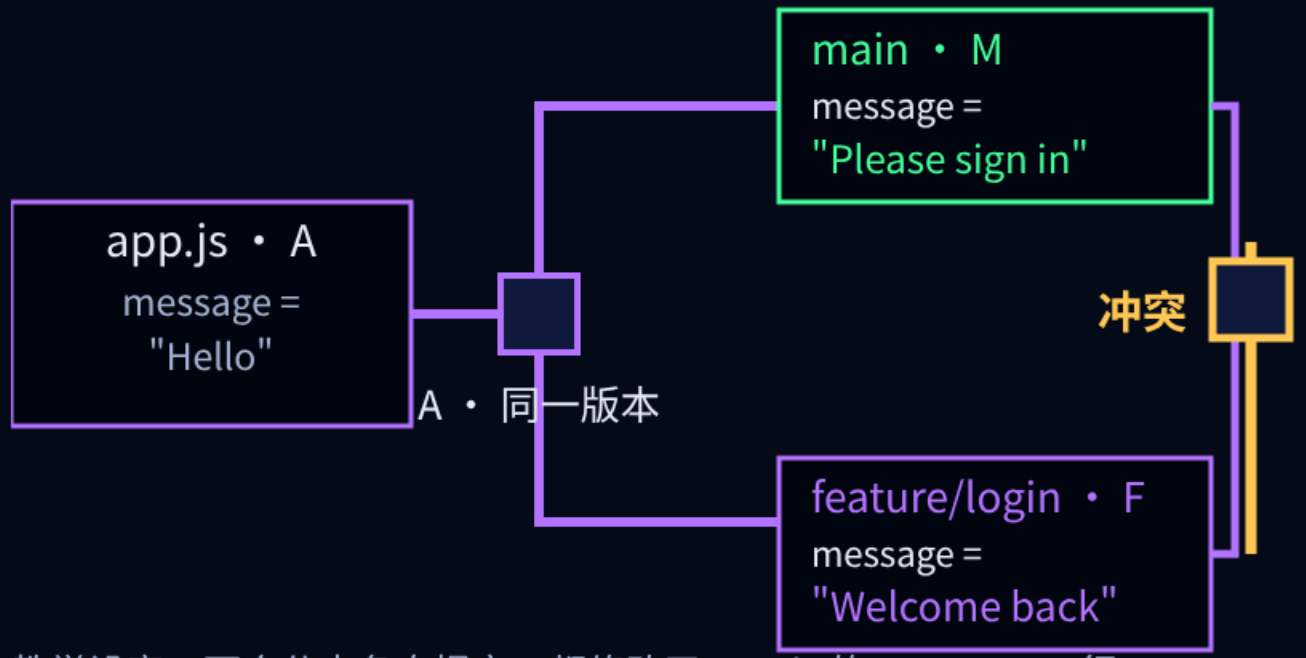
教学示意：未暂存修改尚不在暂存区，也不会进入提交 B。

分支是可移动的标签

HEAD 告诉我们当前所在位置

```
git status --short --branch
git log --oneline --graph --decorate
```

同一行改动，让合并停下



教学设定：两个分支各自提交，都修改了 app.js 的 message 一行。

Git 为什么不能替我们选？

互不影响的改动可以合并。
同一个决定的两种答案，
需要人来确认。

LEVEL 01

冲突来自双方的重叠改动

在 main 执行合并；双方都从共同祖先 A 修改了 app.js 的同一行。

A · 共同祖先

初始化应用

shop-app/app.js

```
let message =
"Hello";
```

main · M

加入登录提示

shop-app/app.js

```
let message =
"Please sign in";
```

feature/login · F

加入欢迎语

shop-app/app.js

```
let message =
"Welcome back";
```

```
$ git merge feature/login
```

同一行有不同结果 → **需要人来决定**

改动不重叠，Git 可以自动合并；同一处改动不同，自动选择可能改变程序行为。教学节选

LEVEL 01

读懂冲突标记的三段内容

在 main 执行 git merge feature/login 后，打开 shop-app/app.js

app.js · 冲突区域　　教学节选

```
function greet(user) {
<<<<<<< HEAD
let message = "Please sign in";
=======
let message = "Welcome back";
>>>>>>> feature/login
return message + ", " + user + "!";
}
```

当前分支的版本

这里是执行合并时的 **main**，不是永远叫 Alex。

分隔两段候选内容

上面选哪段、下面选哪段？也可以整合双方。

正在并入的版本

这里是 **feature/login**；反转合并方向会换边。

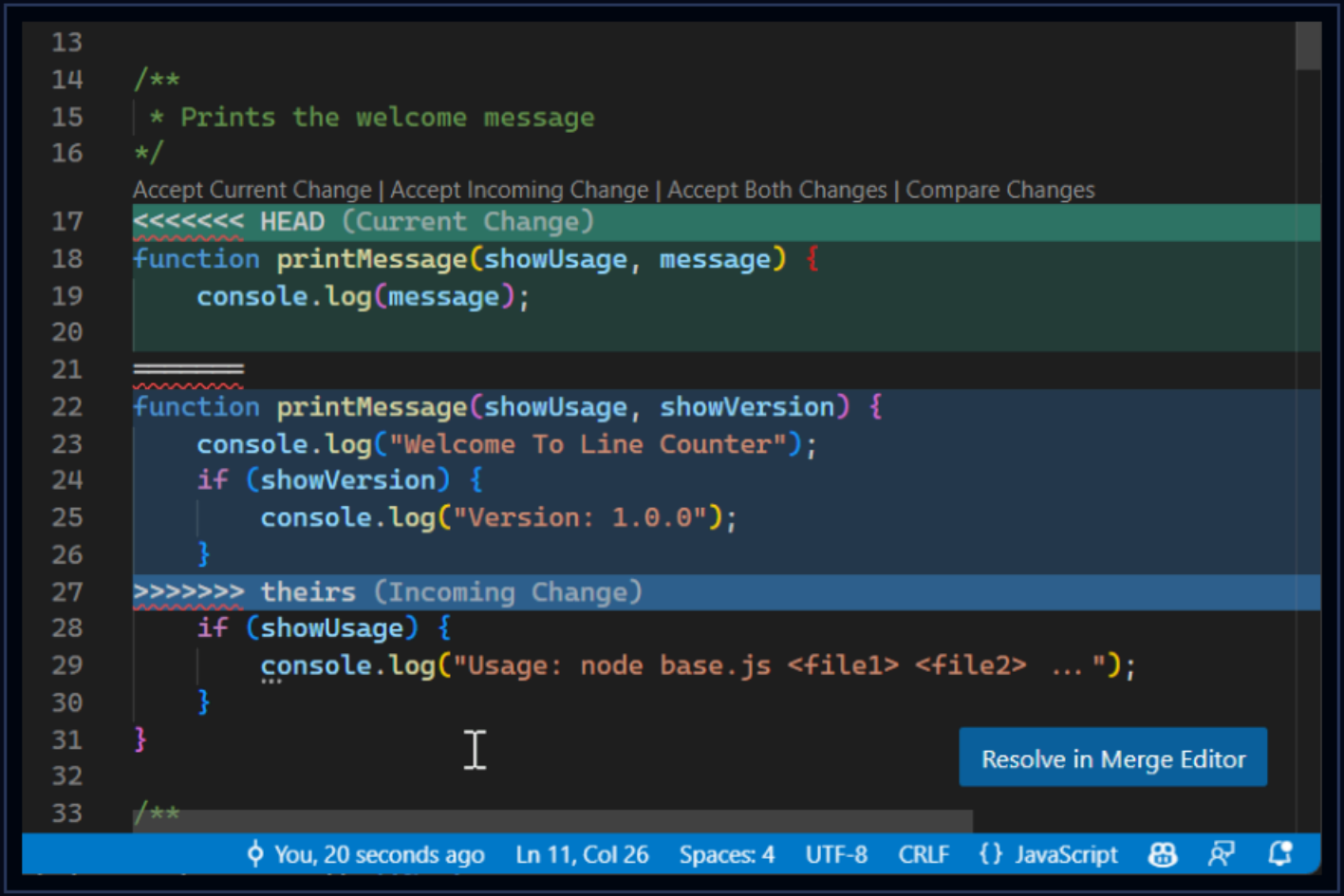
先理解两个意图，再编辑最终内容；冲突标记不是要保留的代码。

教学整合示例：message = "Welcome back; please sign in"；　这不是唯一业务答案。

LEVEL 01

用真实 IDE 定位并处理冲突

在终端发现冲突后，打开 IDE 查看同一文件，定位候选改动并明确完成解决。



VS Code 官方文档截图 · 署名 Microsoft · CC BY-4.0 · 改编说明：仅裁去网页视口黑边，未修改界面内容

01 **定位冲突**

冲突区域被高亮分段，`<<<<<<< HEAD (Current Change)` 与 `>>>>>>> theirs (Incoming Change)` 标出两侧候选。

02 **选择或编辑最终内容**

用 **Accept Current Change** / **Accept Incoming Change** / **Accept Both Changes** 快速选择，或直接编辑；也可 **Compare Changes** 对比、**Resolve in Merge Editor** 进入合并编辑器。

03 **标记已解决**

逐段处理完所有冲突后，文件不再含冲突标记，再回到终端用 `git add` 记录结果。

LEVEL 01

从 status 到 resolved：四步走完

当前在 main，合并 feature/login 后，app.js 未合并。

01

识别冲突

```
$ git status

You have unmerged paths.
both modified:
  app.js
```

unmerged

教学节选
每步之后都用 status 验证。

02

编辑并做决定

```
app.js
let message =
  "Welcome back;
   please sign in";
```

已编辑

整合双方意图，
删除全部冲突标记。

03

记录解决结果

```
$ git add app.js

$ git status

All conflicts fixed
but you are
still merging.
```

added

教学节选
文件已暂存，合并尚未完成。

04

完成合并提交

```
$ git commit

G 合并登录功能
父提交：M + F
```

merged

再查 status 与
`log --graph`。

> `git add` 是记录解决结果，不是魔法修复。

官方也提供 git merge --continue；本页主流程使用 git commit。

LEVEL 01

保留双方，不等于随意删标记

教学示例：检查 shop-app/app.js 的最终内容，而不是只看命令有没有报错。

错误：留下冲突标记

教学示例

app.js · 冲突区域

```
<<<<<<< HEAD
let message = "Please sign in";
=======
let message = "Welcome back";
>>>>>>> feature/login
```

只删标记、丢掉必要逻辑，同样可能出错。

标记必须清除，代码意图也必须正确。

正确：整合后清除全部标记

教学示例

app.js · 同一区域

```
let message = "Welcome back;
please sign in";
return message + ", " + user + "!";
```

本例保留登录提示与欢迎语。

这是教学业务决定，不是唯一答案。

再检查标记

教学建议

```
$ git grep -n '^<<<<<<<' --
- app.js
```

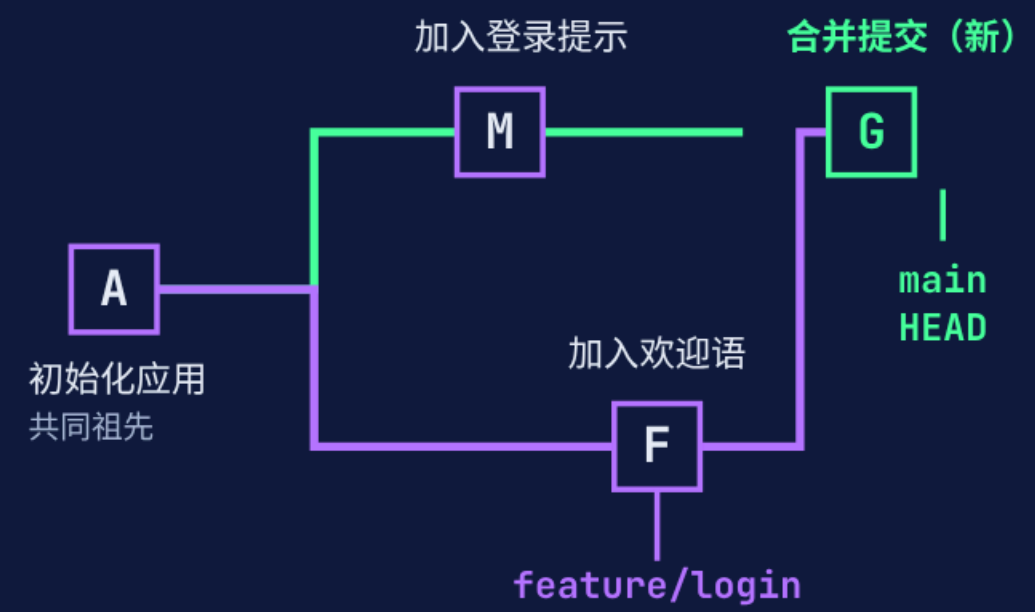
无匹配时不显示冲突行；也可在 IDE 确认全部冲突已解决，再运行项目适用的测试。

LEVEL 1 · 合并冲突

合并完成后，历史长这样

shop-app · app.js · 教学示例：在 main 上执行 git merge feature/login，解决冲突后提交

git log --graph --oneline --decorate · 教学节选



G 的两个父提交：M 和 F

app.js · 解决冲突后的最终内容

```
function greet(user) {
  let message =
    "Welcome back; please sign in";
  return message + ", " + user + "!";
}
```

双方改动都保留了 —— 这是示例的业务决定，不是唯一答案。

git status · 教学节选

```
On branch main
nothing to commit, working tree clean
```

合并提交 G 已完成，分支指针移动到 G。

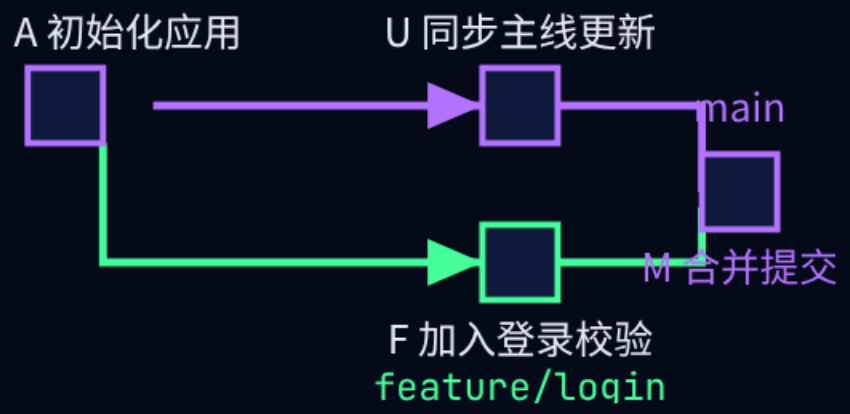
合并提交把两条线连接起来：它有两个父提交，普通 merge 不改写、不覆盖对方的旧提交。

LEVEL 02 · REBASE

整理分支前，先比较两种历史

merge：保留分岔与汇合

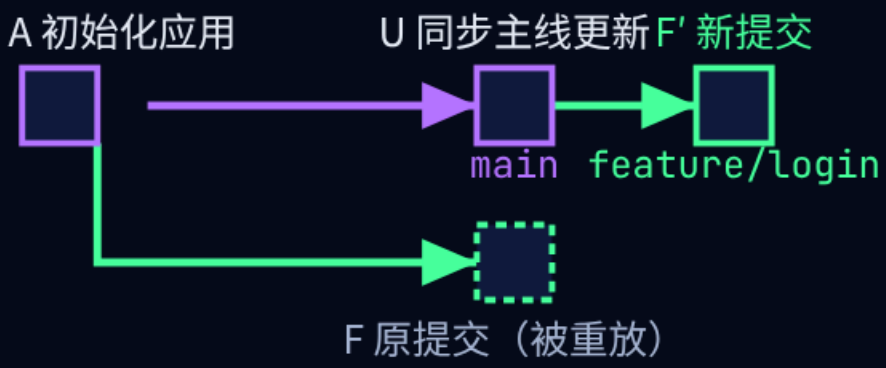
在 main 执行 git merge --no-ff feature/login



新增合并提交 M，分岔原样保留

rebase：把提交接到主线之后

在 feature/login 执行 git rebase main



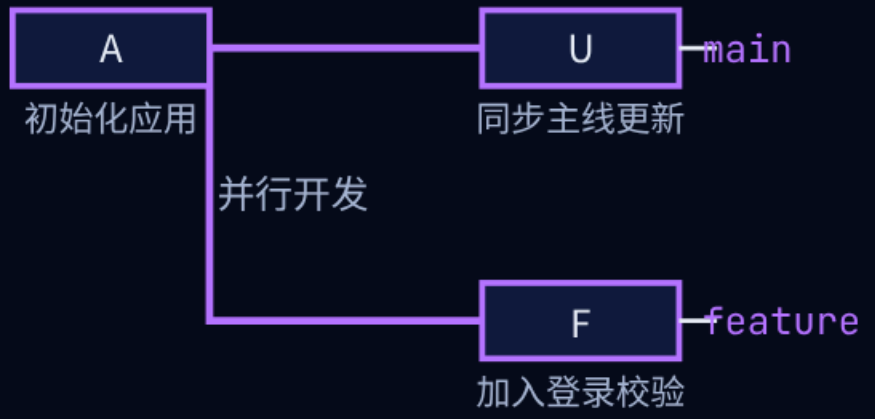
F 重新应用到 U 之后，得到新提交 F′

？同一任务，两种选择：保留分岔，还是整理自己的分支？

merge 保留分岔，也保留汇合

切到 main，把 feature/login 并入　`git merge --no-ff feature/login`

操作前



当前分支 main

`git merge --no-ff feature/login`

→

新增 M

操作后



M：合并登录功能

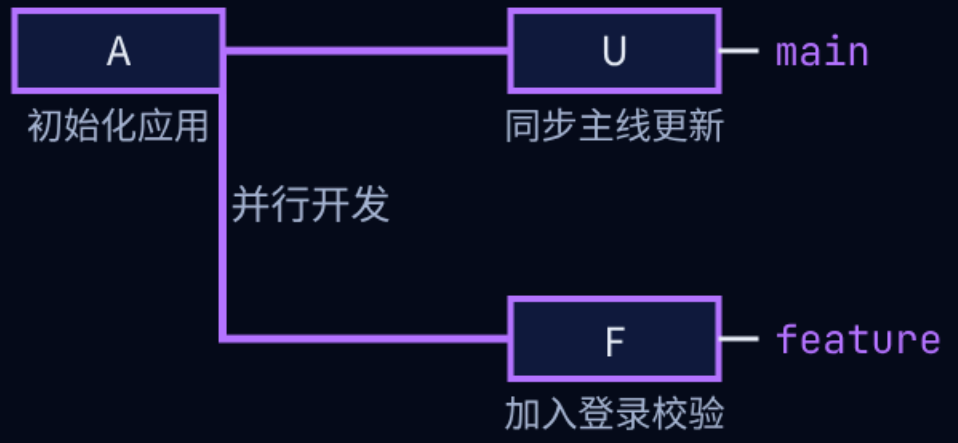
U、F 都留在原处；M 有两个父提交，分岔与汇合都保留。

教学拓扑示意 · 图中 feature 即 feature/login；绿框：新增提交；竖线：分支指针

rebase 把新工作接到目标分支之后

切到 feature/login，把 F 接到 main 之后　`git rebase main`

操作前



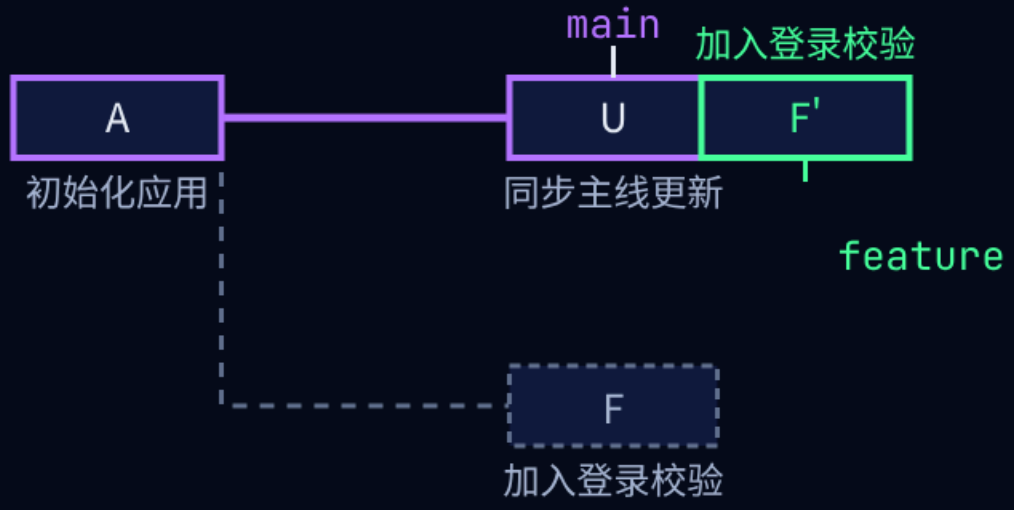
当前分支
feature/login

`git rebase main`

→

重新应用 F

操作后



F'：相同改动，通常有新提交标识

不是把旧提交搬个位置，而是重新应用改动；feature/login 改指 F'。

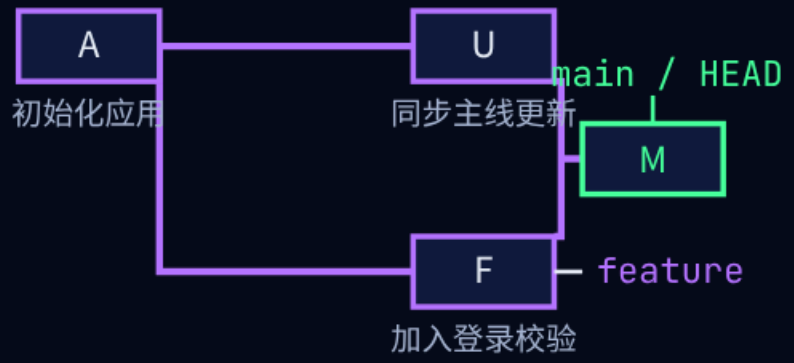
教学拓扑示意 · 图中 feature 即 feature/login；虚线：旧 F 不在分支尖端，并非对象立即删除

同样合并，两种历史如何不同

相同输入：main 有 U，feature/login 有 F；双方改动都保留。

merge：新增汇合提交 M

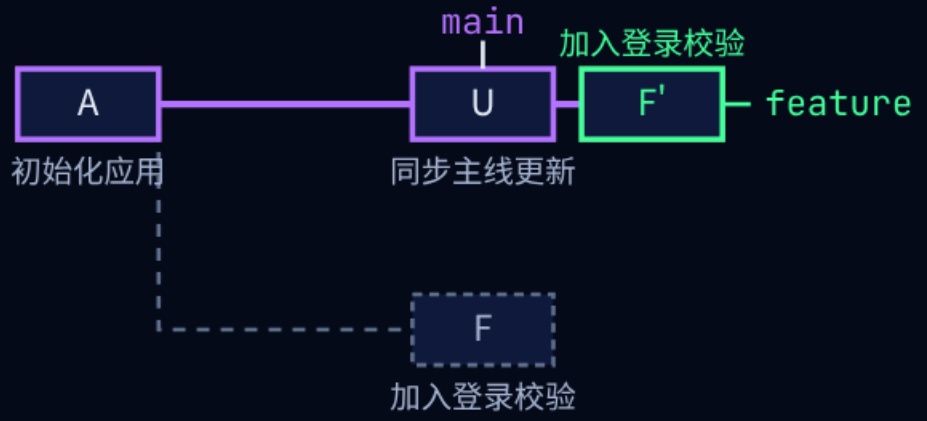
在 main 合并；feature/login 仍指 F



保留分岔与汇合

rebase：重新应用为 F'

在 feature/login 变基；main 仍指 U



改动可以都保留，历史却不同：是否改写原分支历史，取决于共享状态与团队约定。

教学拓扑示意 · 图中 feature 即 feature/login；绿框：操作新增；虚线：旧 F 不在分支尖端

LEVEL 02 · REBASE

未推送的整理，已推送的谨慎

提交是否已共享？

别人是否已拉取、或基于它开发？

否 · 仅本地

可考虑 rebase

典型场景：**整理自己的功能分支、把主线更新同步到新工作**。没有协作者依赖旧历史，改写只影响自己。

是 · 已推送

不要擅自改写

他人可能基于旧基线开发，改写会让双方历史意外分叉，带来**协作协调成本**。确需改写：先沟通、先 `fetch`，按团队规则考虑 `--force-with-lease`；lease 失败不要连续 `--force` 绕过。

> **--force-with-lease 不是万能的：**它只校验远端引用与本地 remote-tracking 记录一致，不是实时协作锁，也不能证明没人拉取过旧提交。

LEVEL 02 · REBASE

变基遇到冲突：先解决，再继续

在 feature/login 执行 git rebase main
重放到 F 时暂停，等待解决冲突



shared.js 冲突：HEAD 侧 = 新基线 main 的改动
下侧 = 正在重放的 feature 提交（不能机械等同分支名）

冲突不是失败，是**暂停等待你的决定**。多个提交可能逐个暂停，逐个解决，不能一次性 add 所有未解决文件就算完成。

01 解决冲突

编辑 shared.js，整合双方改动

02 暂存当前文件

```
git add shared.js
```

03 继续变基

```
git rebase --continue
```

让流程继续重放剩余提交

或 **中止变基**

```
git rebase --abort
```

分支回到变基开始前的状态，已提交的工作仍在原处

注意：abort 只是放弃这次变基、回到变基前状态，不是丢弃所有未推送工作的通用命令。操作前先确认当前分支和想保留的提交。

LEVEL 03

换任务紧急，半成品怎么办？

工作区还有改动

需要临时切换分支



LEVEL 03

stash 把改动从当前工作区借走

教学示例 · feature/login

$ git stash push -m "登录：未完成校验"

已跟踪的 app.js 有未提交修改。
执行后，工作区与索引回到 C1。

stash push ≠ git push
保存到本地 stash，不推送到远程。

默认不保存未跟踪新文件
需要时加 -u / --include-untracked，仍不包括 ignored。



改动进入独立 stash 栈
不是提交，也不是远程备份。

LEVEL 03

列出、查看，再决定何时取回

先确认是哪一份改动

```
$ git stash list
stash@{0}: On feature/login:
登录：未完成校验
$ git stash show -p stash@{0}
```

教学节选 · `show -p` 查看改动补丁。
确认索引和内容后，再选择取回方式。



栈条目：保留
取回后仍需明确 drop

成功：删除条目
冲突：记录仍保留

`git stash apply stash@{0}`
像复制回来：改动恢复，记录仍在。

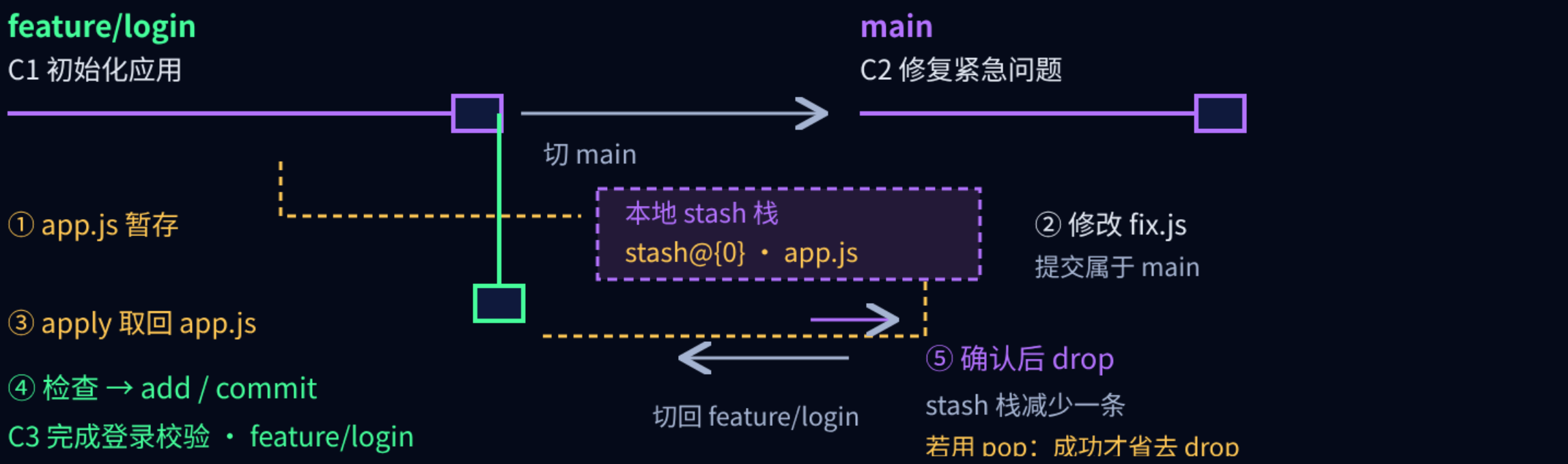
`git stash pop`
尝试取走并删除；**不能假设冲突后已删除。**

默认恢复不保证重建原暂存状态；`git stash drop` 明确删除。初学者不要随意 `clear`。

LEVEL 03

回到原任务，让改动安全落地

教学闭环 · 跟随琥珀色 app.js 改动，提交不会自动搬家




```
git stash push -m "登录：未完成校验"
```

先保存工作，再切分支。

```
git switch main → 修改 fix.js
git commit -m "修复紧急问题"
```

```
git switch feature/login
git stash apply stash@{0}
```

检查 app.js 后 add / commit。

LEVEL 04

status 正常，不代表站在分支上

检出旧提交查看后，git status 可能显示：`Not currently on any branch.`



`git checkout 7d2b4e8`　情境示意 · 教学哈希

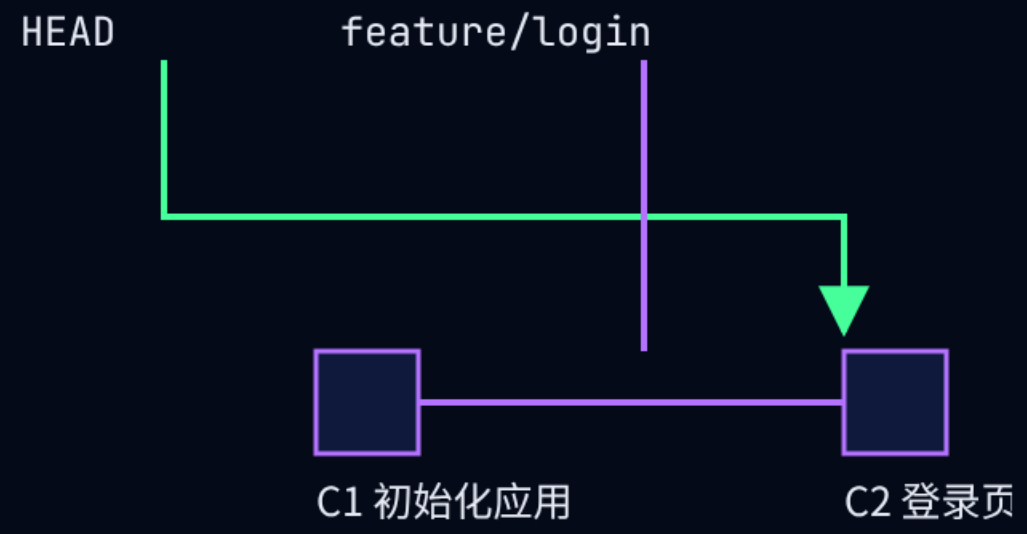
在这里新建的提交，会回到原分支吗？

LEVEL 04 / 游离 HEAD

detached HEAD 是 HEAD 直接指向提交

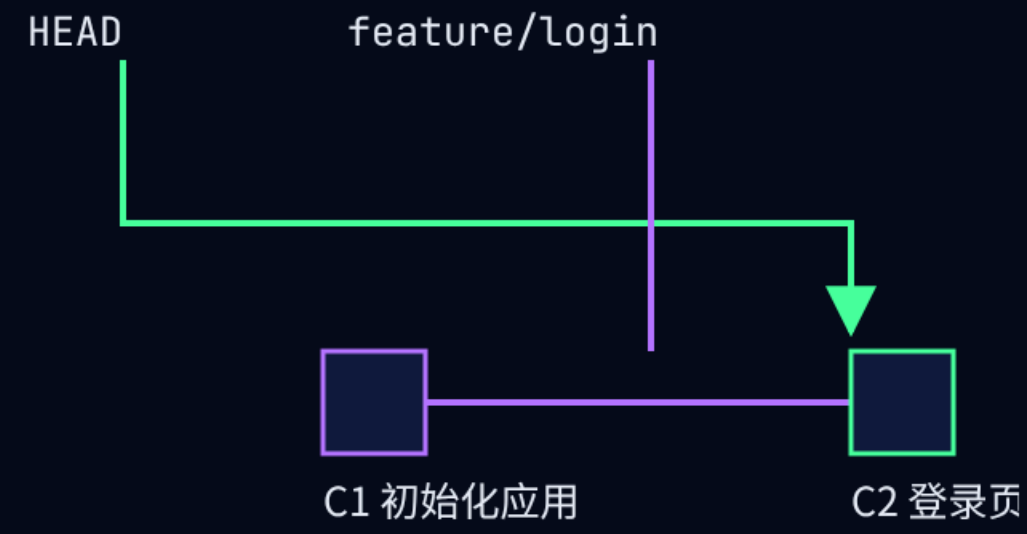
在分支上

HEAD 经过分支名，新提交可推动分支前进



游离 HEAD

HEAD 跳过分支；feature/login 仍停在 C1



常见成因：`git checkout C2` / `HEAD~1` / `v1.2.0` / `origin/main`

不是文件丢失，也不是提交立刻消失。

教学拓扑：feature/login 为 C1→C2；右侧执行 `git checkout C2`。

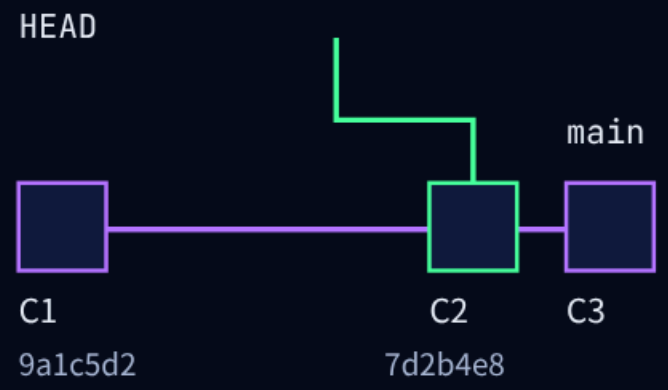
LEVEL 04 / 游离 HEAD

用 branch --contains 判断位置

```
$ git status -b
Not currently on any branch.
nothing to commit, working tree clean

$ git branch --show-current
（无分支名输出）

$ git branch --contains HEAD
* main
```



main 包含 C2

因为 C2 是 C3 的祖先。

不代表正在 main 工作。

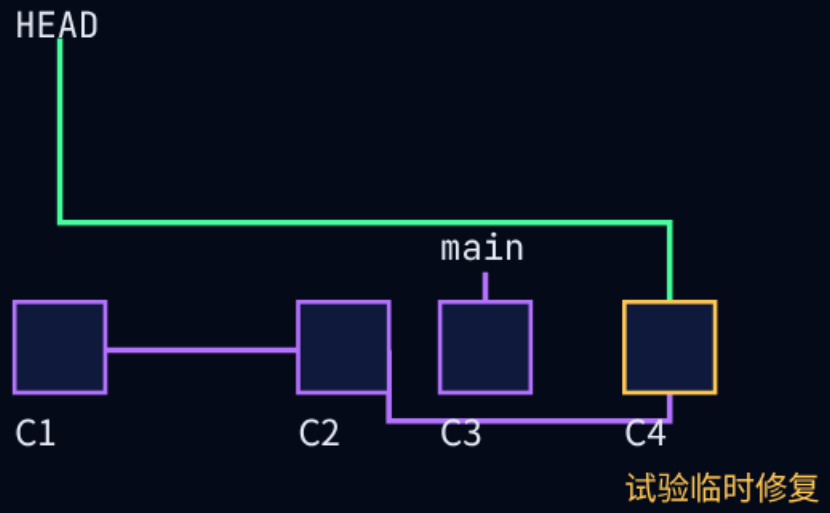
教学节选 · 虚构哈希
若已新增 C4，main 不包含 C4。

先确认“是否游离”，再确认“当前提交由哪些分支保护”。

LEVEL 04 / 游离 HEAD

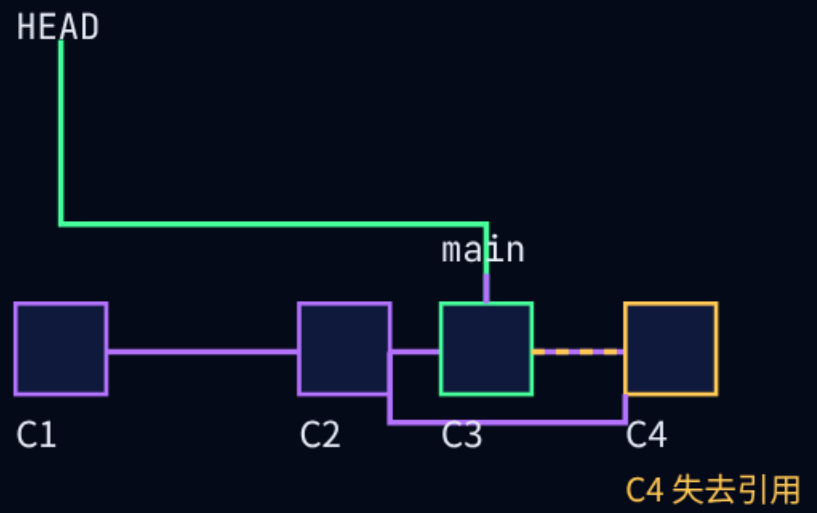
新提交不会自动跟着分支走

在游离的 C2 上提交 C4



HEAD→C4；main 仍→C3。

随后 git switch main



HEAD 回到 main 的 C3；C4 不再受其保护。

离开前先保护 C4，不是等切走后再找。

提交不会马上从对象库消失，但可能难以找到；“切走”不等于“永久删除”。

离开前先建分支，再记下提交号

在旧提交上有需要保留的新提交吗?

没有新提交，且工作区干净 A

$ git switch main

C2 本来就在 main 历史中，无需另建空分支。



已有新提交 C4「试验临时修复」 B

$ git switch -c rescue/temporary-fix → git switch main

先让分支指向 C4，再离开；新工作不会跟着 main 走。



额外保险　离开前记录完整提交号

```
git rev-parse HEAD
```

还有未提交修改?　先判断归属，决定 commit / stash。

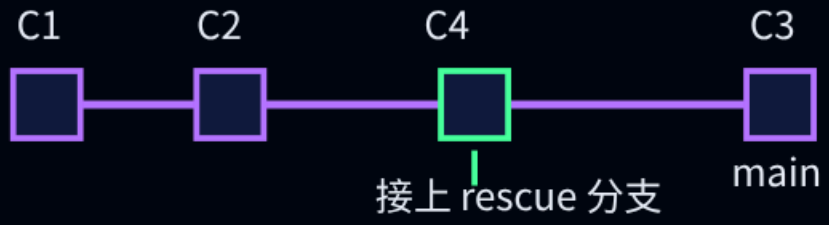
相同文件内容不同，切换可能失败。

已经切走了，用 reflog 找回

曾在 C2 提交 C4，但未建分支就切到 main。目标：让 C4 重新获得稳定引用。

`git reflog` ／ 教学节选，实际提交号不同

```
0f6c2a9 HEAD@{0}: checkout: moving from
detached HEAD to main
a41d7c0 HEAD@{1}: commit: 试验临时修复
7d2b4e8 HEAD@{2}: checkout: moving
from main to 7d2b4e8···
```



01 **查找目标**

```
git reflog
```

按提交说明找 C4，取实际完整哈希。

02 **核对，再建立保护**

```
git show --stat --oneline <实际完整哈希>
git branch rescue/temporary-fix \
  <实际完整哈希>
```

03 **返回并验证**

```
git switch main
git log --decorate rescue/temporary-fix
git status
```

确认分支指向 C4，工作区符合预期。

reflog ≠ 永久备份　它是本地近期引用记录，不跨同事共享；编号会变化，不能照抄 HEAD@{1}。

这不是恢复被覆盖的文件；找到后尽快验证，并按团队方式备份。未提交改动不在本节承诺范围。

把四关放回同一个协作流程

先描述状态，再选工具：改了什么？在哪个分支？是否已提交、已共享？

双方改动重叠

同步 / 整理
未共享分支

临时切任务
工作未完成

查看旧提交
detached HEAD

01 合并冲突	02 merge / rebase	03 stash	04 确认与保护
编辑冲突并完成 merge 决定最终文件内容	比较历史与风险 共享历史优先保护	暂存后切走 回来取回并提交	判断是否有新提交 必要时建分支

每关都验证 → 文件内容 `git status` 分支指针 `git log --graph` 必要时 `git reflog`

现场演练：从冲突到取回改动

在本地练习仓库操作，不直接改写团队共享分支。

教学示例 · 提交号虚构

01 **合并双方对同一行的改动**

main 与 feature/login 同改 app.js 的 message 行；在 main 合并并解决冲突。

```
git merge feature/login → 编辑 → git add app.js → git commit
```

检查：冲突标记已处理，最终内容符合决定。

02 **暂存登录改动，完成另一任务**

在 feature/login 暂存未提交修改，切 main 提交 fix.js，再切回取回。

```
git stash push -m "登录：未完成校验" → 切 main 提交 fix.js → 切 feature/login → git stash apply → git commit
```

检查：app.js 改动已回来；确认后再删除 stash 记录。

03 **从旧提交安全离开**

检出 C2 后用 status 判断；若有 C4，先建分支再返回 main。

```
git checkout 7d2b4e8 → git status
git switch -c rescue/temporary-fix → git switch main
```

检查：status 是否游离；rescue 分支是否指向要保留的提交。



先预测，再执行。

每步先写下预期的 status 与 log 变化，再操作验证。

情境插画，不代表仓库状态。

出问题时，按状态选下一步

$ git status　**先停止继续改写，再判断当前流程**

有未合并冲突	→	编辑文件 → `git add <文件>` 合并：`git commit` 变基：`rebase --continue`	→ 核对内容 再看 status
有未提交改动 且需要切换分支	→	先决定改动归属 `git commit` 或 `git stash push`	→ 确认工作区 再切换分支
变基中方向错误	→	放弃本次变基 `git rebase --abort`	→ 分支回到变基前 核对 status / log
detached HEAD 且已有新提交	→	先建分支保护当前提交 `git switch -c rescue/<分支名>`	→ 已切走？用 `git reflog` 核对后再建引用

以 status 提示的流程为准，不把所有问题都用 continue 解决。

merge --abort ≠ rebase --abort；
不确定共享影响，先询问。

操作前确认，操作后验证

操作前：知道起点与退路

01 看 status，识别当前流程
02 确认分支与改动归属
03 确认提交是否已共享
04 知道如何中止或保护



操作后：核对实际结果

01 看文件最终内容
02 看 status 的状态与提示
03 看 log --graph 的提交关系
04 需要时，用 reflog 查找

带着检查结果，开始下一轮操作

`git status`	`git log --oneline --graph --decorate`	`git branch --contains HEAD`	`git reflog`
当前状态	提交关系与引用	哪些分支包含此处	本地查找入口

没有一条命令能诊断一切；把文件、状态、分支和历史放在一起看。

带着状态意识进入团队协作

先在本地练习仓库完成四关，

再触碰共享分支。

解决冲突 | 保护历史 | 取回改动 | 建立引用

